SOURCE CODE MANAGEMENT TOOL (GIT)

SOURCE CODE MANAGEMENT

it is used to manage all the codes and their versions.

There are two types of method to manage

1. centralised version control system
2. distributed version control system

CENTRALISED VERSION CONTROL SYSTEM:-(CVCS)

1. In centralised version control system, the coders commit their code on central location i.e on repository.
2. when code commit on the repo, we can update the code means another user can access this code.
3. repository is a central location where we can store the code.

Drawbacks

1. it is not locally available meaning you always need to be connected to the network to perform any action. since everything is centralised, if central server gets failed, you will lose the entire code that you can commit on the repository. eg.SVM tool
2. Single point of failure
3. Limited offline access
4. Slow performance
5. Limited branching and merging capabilities
6. dependency on the central server
7. Limited scalability

DISTRIBUTED VERSION CONTROL SYSTEM:-(DVCS)

1. In distributed version control system, the users can commit their data on the local repo first. then push the data on central repository.
2. so that the users have two copies of the code whether the central location should be down or not accessing we can access code from our local repo.
3. In distributed version control system, every contributor has a local copy or clone of the main repository i.e. everyone maintained local repository of their own which contains all the files and metadata present in main repository.

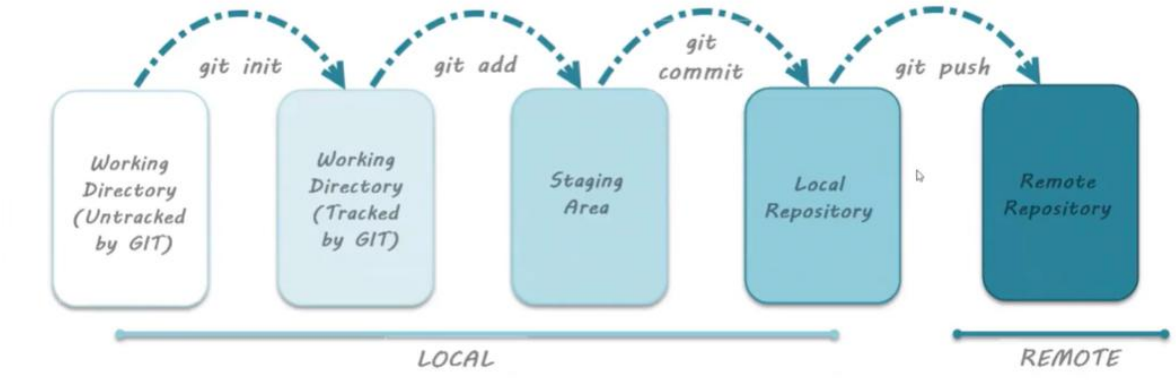
GIT

1. it is a free & open-source distributed version control system where it can store all the different types of codes and maintain their versions.
2. Git is a free and open-source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.
3. it was designed by Linus Torvalds in 2005 and has one of the most widely used version control systems in the software development in industry.

ADVANTAGES OF GIT

1. free and open source
2. fast and small
3. security
4. easier branching
5. no need of powerful hardware's
6. Offline Access
7. Integration with Other Tools
8. Speed and Performance
9. Distributed Version Control

GIT ARCHITECTURE



GIT LIFECYCLE

1. git init
2. git add
3. git commit
4. git push

GIT COMMANDS (install and setup git bash)

Steps to Create files & versions and push it to the remote repository

1.mkdir studentapp
2.cd studentapp
3.git init
4.vim hello.txt
  Hello
5.git status
6.git add .
6.git commit -m “v1”
7.git log
  V1 ----------> commitID:385u48th

Create another version

1.vim hello.txt
  Hiii
2.git add .
3.git commit -m “v2”
4.git log
  V1 ----------> commitID:385u48th
  V2 ----------> commitID:38545vdfgfg

Push it into remote repository

1.create a repository on the github (studentapp)
2.git remote add origin (repo_url)
3.git push origin master
4.git remote -v
5.git push -u origin master

Note:

1. give the authorization access to access to github repository
2. when u are in the local it means u have the ----> master branch
3. when u are in the remote repo u have the -----> main branch

---